

Industrial Internship Report on,

"Design and Development of Python-Based CLI Utility Applications: Quiz Game, File Organizer, URL Shortener, and Password Manager"

Prepared by,

Sumaya Afrin A

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on Python programming, and I had to complete four practical projects along with this report within 6 weeks' time.

My project work involved building a suite of four independent Python command-line utility applications, a Quiz Game, a File Organizer, a URL Shortener, and a Password Manager each designed to automate a common everyday task and to strengthen my understanding of core Python concepts such as loops, conditional statements, functions, dictionaries, and file-extension based logic.

This internship gave me a very good opportunity to get exposure to industry-style problem solving and to design and implement working solutions independently. It was an overall great learning experience.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About UniConverge Technologies Pvt Ltd	4
2.2	About upskill Campus	8
2.3	Objective	9
2.4	Reference	9
2.5	Glossary	10
3	Problem Statement	11
4	Existing and Proposed solution	12
5	Proposed Design/ Model	13
5.1	High Level Diagram (if applicable)	13
5.2	Low Level Diagram (if applicable)	13
5.3	Interfaces (if applicable)	13
6	Performance Test	14
6.1	Test Plan/ Test Cases	14
6.2	Test Procedure	14
6.3	Performance Outcome	14
7	My learnings	15
8	Future work scope	16

1 Preface:

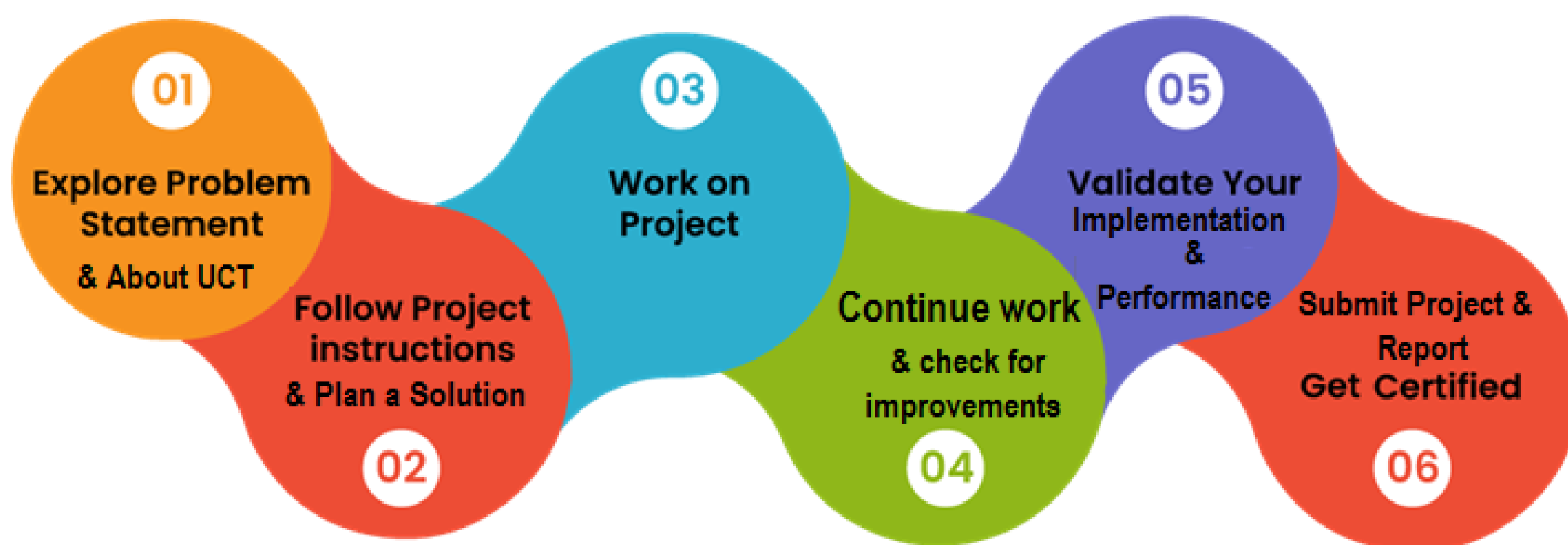
This report summarizes the work completed during the six-week Python Industrial Internship organized by upskill Campus (USC) and The IoT Academy, in collaboration with UniConverge Technologies Pvt. Ltd. (UCT).

Internships play a vital role in a student's career development. They bridge the gap between classroom learning and industry practice, allowing students to apply theoretical knowledge to real, hands-on project work while building confidence, discipline, and problem-solving ability that are difficult to gain from academics alone.

During this internship, I worked on four independent Python projects, a Quiz Game, a File Organizer, a URL Shortener, and a Password Manager. Each project was completed over one week, with a structured weekly progress report documenting the objectives, achievements, challenges faced, and lessons learned.

The program was planned as a weekly, project-based learning track: each week introduced a new problem statement, which I solved independently using Python, tested thoroughly, and documented. This iterative structure helped me steadily build proficiency, moving from basic syntax and control structures in Week 1 to menu-driven, data-structure-based applications by Week 4.

How Program was planned



Over the course of this internship, I gained practical experience in Python fundamentals, file handling, string operations, dictionaries, and modular program design. Beyond the technical skills, I also improved my logical thinking, debugging discipline, and ability to independently research and resolve issues, all of which are directly relevant to a career in software development.

I would like to thank upSkill Campus, The IoT Academy, and UniConverge Technologies Pvt. Ltd. for structuring and providing this internship opportunity, and my faculty at Mangayarkarasi College of Engineering for their continued support and encouragement throughout this program.

To my juniors and peers who will take up similar internships: approach every weekly task as a real problem to be solved, not just an assignment to be submitted. Test your code thoroughly, document your challenges honestly, and use each week to build on the previous one, that incremental habit is what makes six weeks add up to real, usable skill.

1. Introduction:

1.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



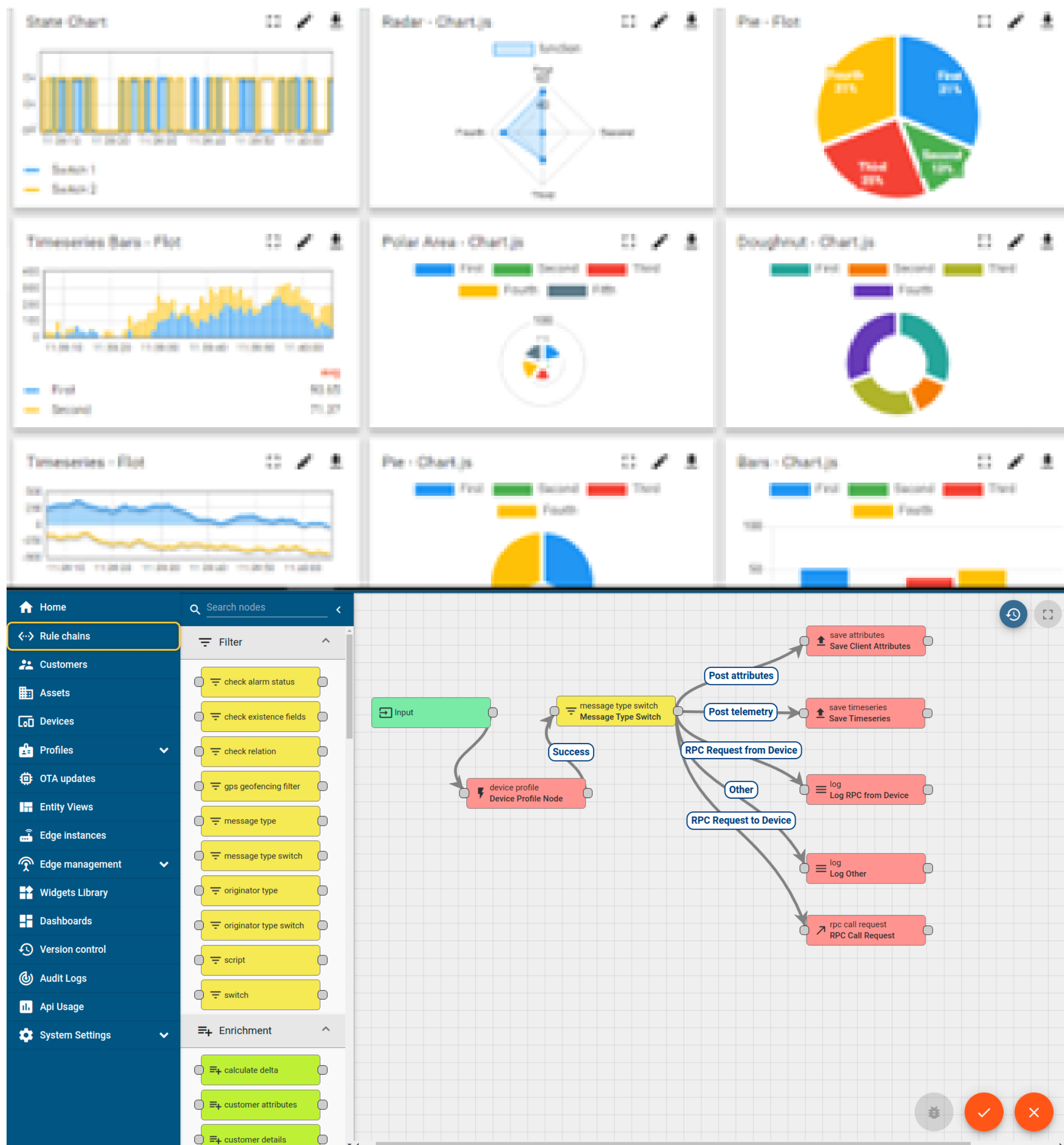
i. UCT IoT Platform()

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



ii. Smart Factory Platform (**FACTORY WATCH**)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

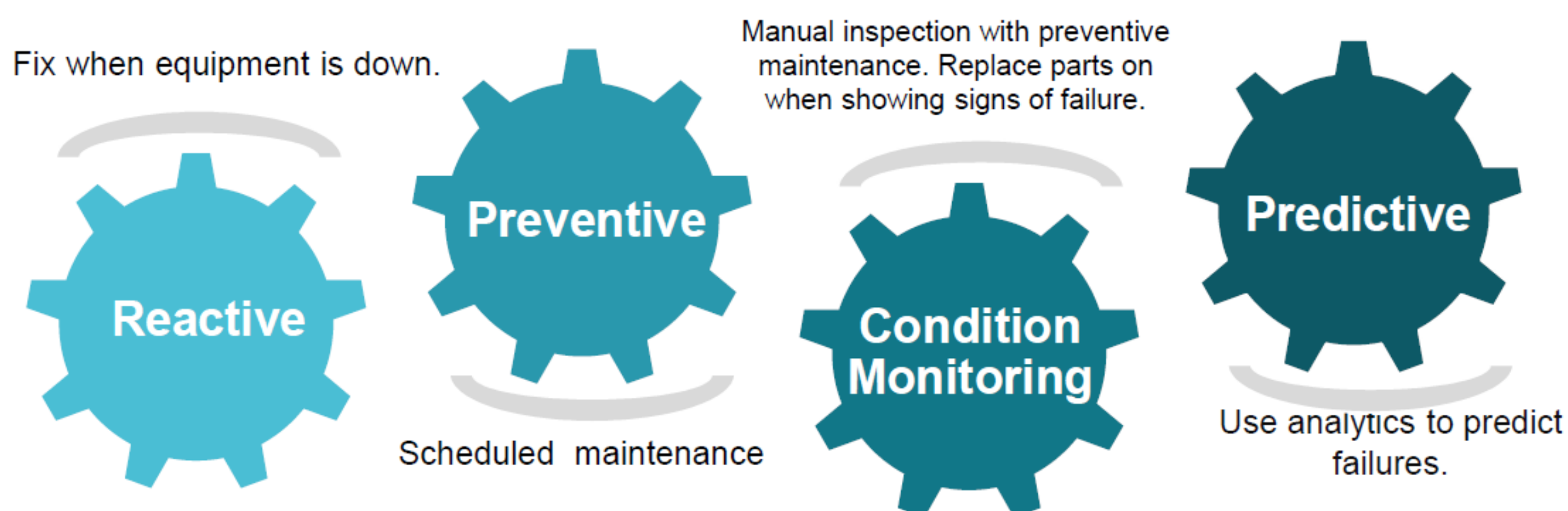


iii. based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

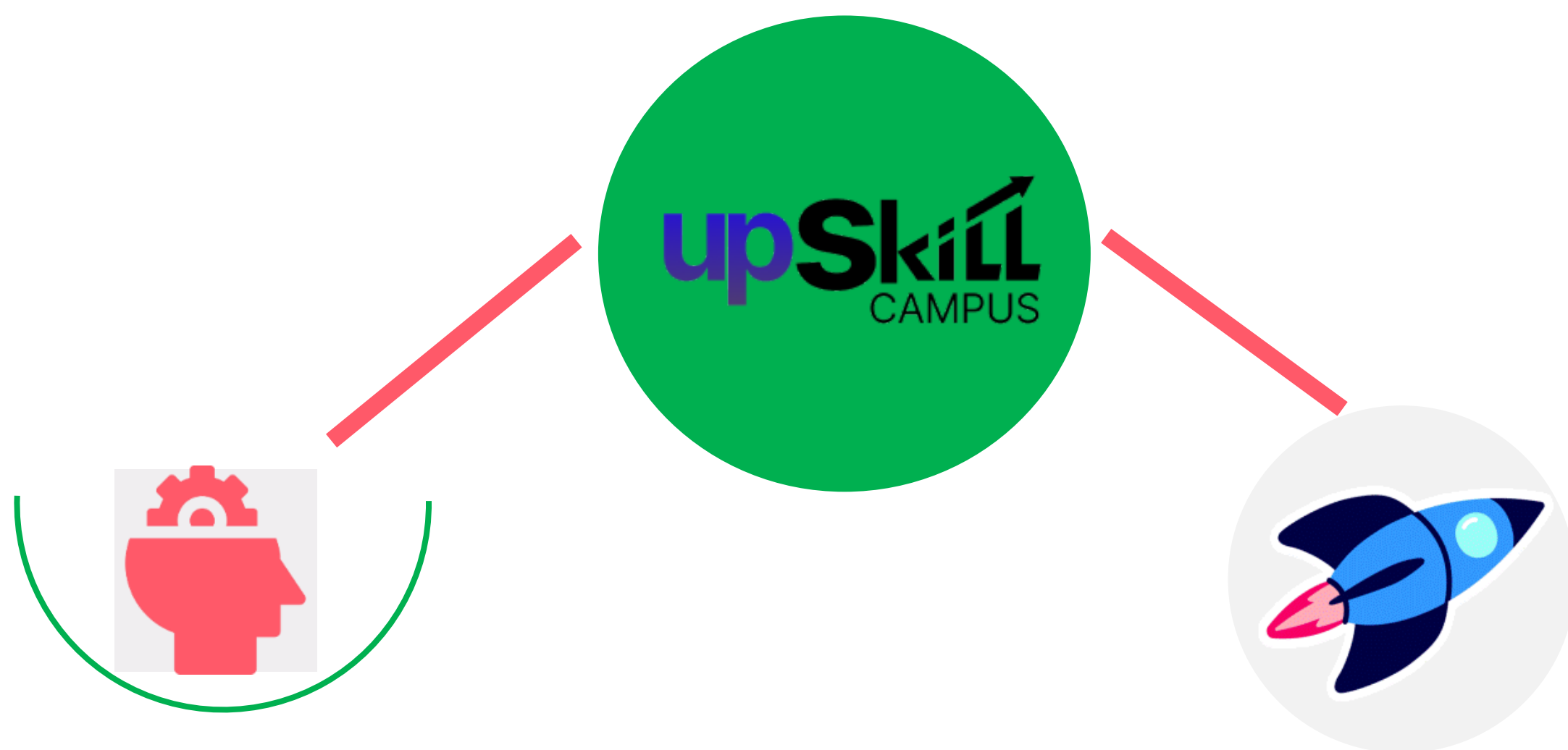
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



1.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

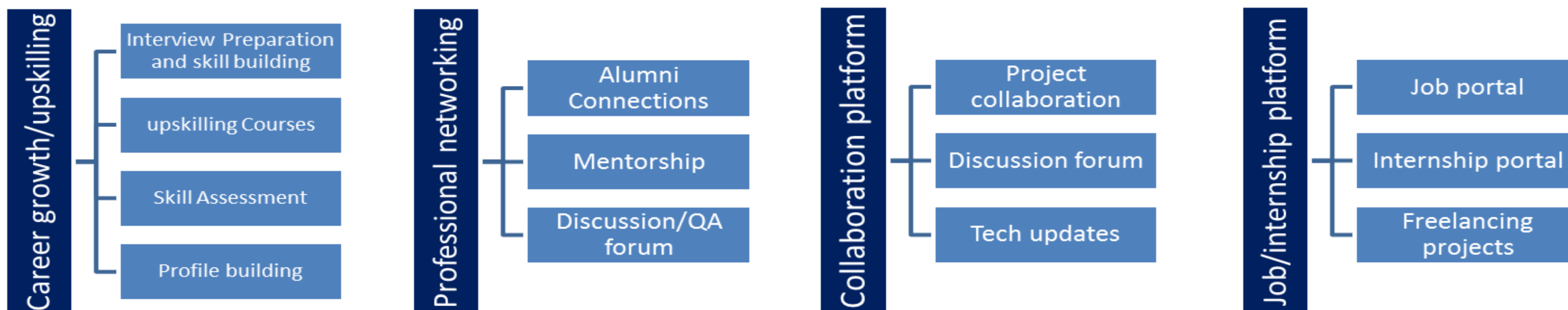
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



1.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

1.4 Objectives of this Internship program:

The objective of this internship program was to:

Get practical, hands-on experience of working in the software industry.

Solve real-world, problem-statement-driven programming tasks.

Improve job prospects through demonstrable, tested project work.

Gain an improved understanding of Python and its practical applications.

Support personal growth better communication, documentation, and problem-solving ability.

1.5 Reference

[1] Python Software Foundation - Official Python Documentation, <https://docs.python.org/3/>

[2] GeeksforGeeks - Python Programming Tutorials, <https://www.geeksforgeeks.org/python-programming-language/>

[3] W3Schools - Python Reference, <https://www.w3schools.com/python/>

1.6 Glossary

Terms	Acronym/Meaning
CLI	Command-Line Interface
IDE	Integrated Development Environment

OOP	Object-Oriented Programming
URL	Uniform Resource Locator
CPU	Central Processing Unit

2 Problem Statement:

In everyday computing and academic use, several small but recurring tasks are handled manually or through heavyweight, internet-dependent, third-party tools:

Self-assessment through quizzes is usually done via online platforms (Google Forms, Kahoot) that require internet access, account sign-up, and cannot be run offline for quick practice.

Files downloaded or saved on a system (images, documents, videos) accumulate in a single folder and must be manually sorted into categories, which is repetitive and error-prone.

Long URLs are difficult to share, remember, or use in space-constrained contexts, and existing shortening services (bit.ly, TinyURL) require an internet connection and account/API access.

Passwords for multiple websites are often written down, reused, or stored insecurely, while full-featured password managers (LastPass, Bitwarden) are third-party services that require trust, installation, and sometimes a subscription.

The assigned problem statement for this internship was to design and build lightweight, standalone Python command-line applications that solve each of these four problems independently without external dependencies, internet access, or paid services while reinforcing core Python programming concepts.

3 Existing and Proposed solution:

Existing Solutions and Their Limitations:

Quiz platforms (Google Forms, Kahoot, Quizizz), require internet connectivity, account creation, and are not usable for quick offline self-practice.

File management utilities (built-in OS sorting, third-party apps like File Juggler) often require installation, licensing, or manual configuration of complex rules.

URL shorteners (Bit.ly, TinyURL) depend on external servers, internet access, and sometimes API keys or rate limits.

Password managers (LastPass, Bitwarden, Dashlane) are feature-rich but require installing third-party software, creating an account, and trusting an external company with sensitive credentials.

Proposed Solution:

Four independent, lightweight Python command-line applications were designed and developed, each addressing one problem statement directly:

Quiz Game - a dictionary-driven quiz engine that asks questions, validates answers case-insensitively, computes a percentage score, and assigns a letter grade, runnable offline in any Python environment.

File Organizer - a rule-based script that classifies a list of files into Images, Documents, Videos, and Others by checking file extensions, and can be extended to operate on real folders.

URL Shortener - a menu-driven application that generates a unique 6-character alphanumeric short code for any URL, stores the mapping in a dictionary, and retrieves the original URL on demand.

Password Manager - a menu-driven application to save, view, generate, and delete passwords per website, using Python's random and string modules to generate strong passwords.

3.1 Code submission (GitHub link):

<https://github.com/asumayaafrin/upskillcampus>

3.2 Report submission (GitHub link):

<https://github.com/asumayaafrin/upskillcampus>

4 Proposed Design/ Model:

All four applications follow the same overall design philosophy: a simple text-based menu presented to the user, input collected through Python's `input()` function, processing handled by dedicated functions or inline logic, and results stored in an in-memory data structure (dictionary or list) before being displayed back to the user via `print()` statements.

4.1 High Level Diagram:

Figure 1: High-Level Architecture of the Python Utility Suite

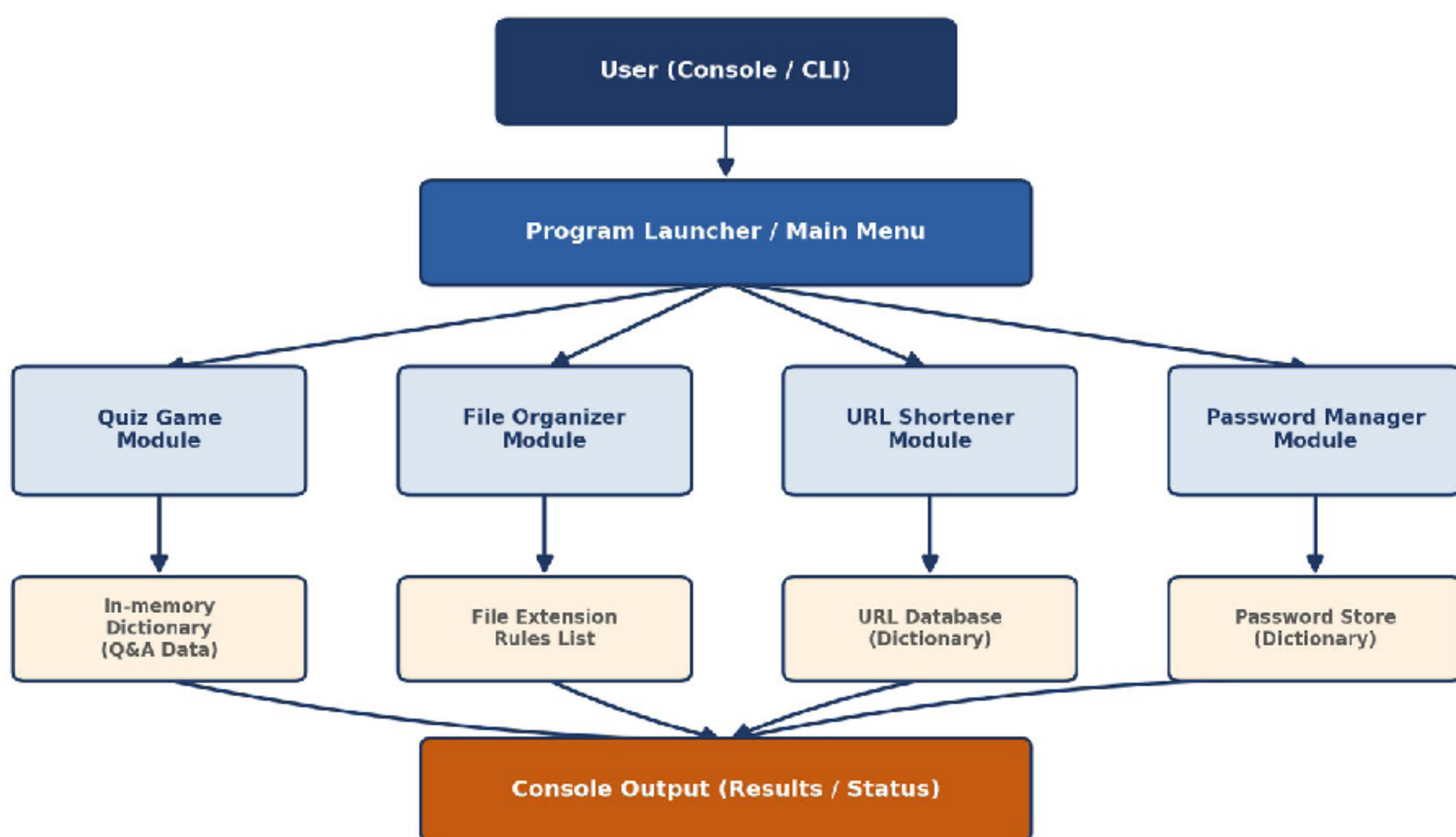


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

The Program Launcher acts as the entry point that directs execution to one of the four independent modules. Each module maintains its own lightweight in-memory data store, a dictionary for the Quiz Game's Q&A pairs, a rule list for the File Organizer, a dictionary for the URL Shortener's mappings, and a dictionary for the Password Manager's credentials.

4.2 Low Level Diagram:

Figure 2: Low-Level Program Flow (Common to All Modules)

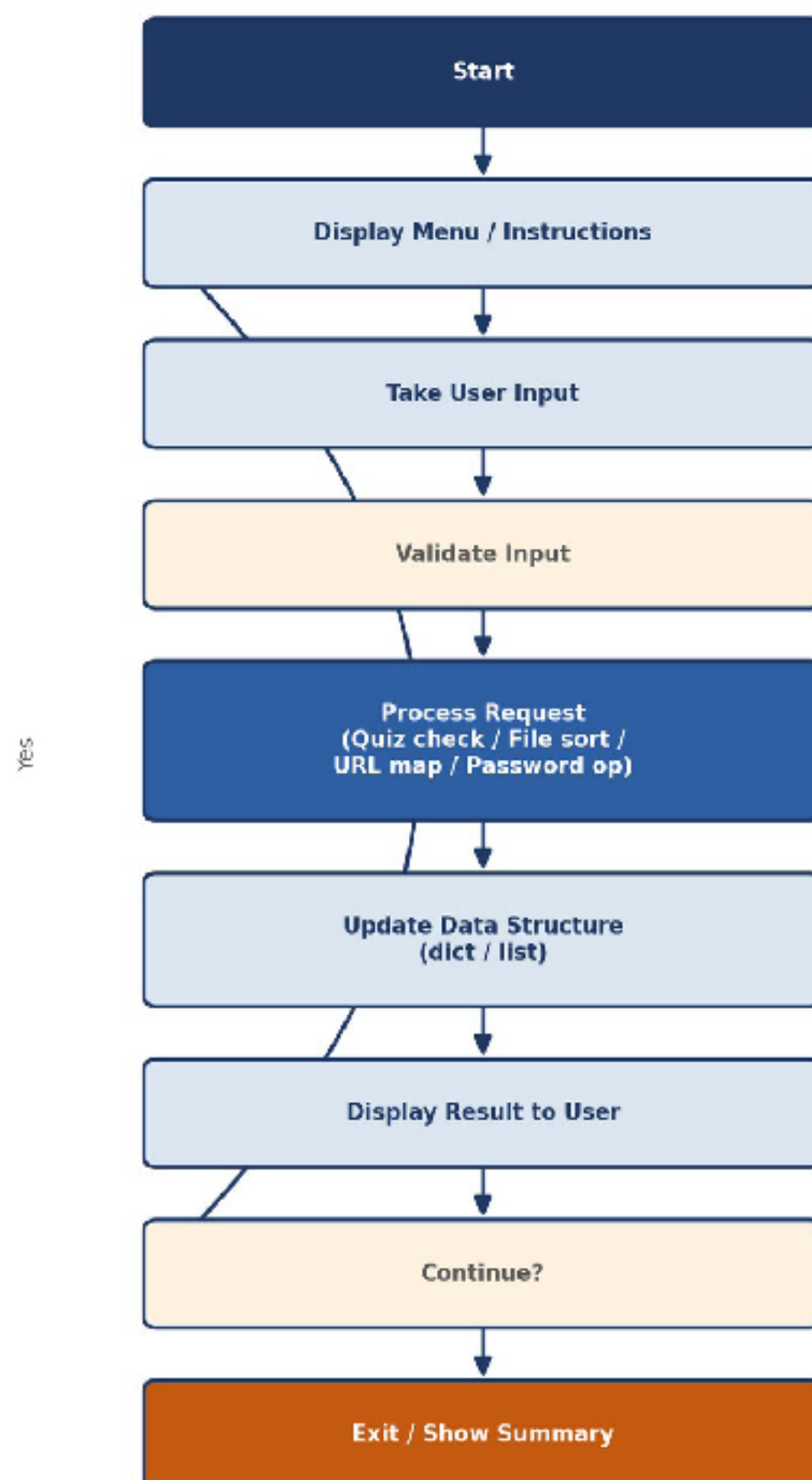


Figure 2: LOW-LEVEL PROGRAM FLOW (Common to All Modules)

This control flow is common to the URL Shortener and Password Manager (which are menu-driven and loop until the user exits), while the Quiz Game follows the same steps once per question, and the File Organizer runs the process/update/display steps once per file in the input list.

4.3 Interfaces:

Figure 3: User-System Interface Diagram (CLI-Based Interaction)



Figure 3: User-System Interface Diagram (CLI-Based Interaction)

All four applications use a plain-text Command-Line Interface (CLI). The user interacts entirely through the console: `input()` calls collect choices and data, and `print()` calls display menus, prompts, and results. No graphical interface, file storage, or network interface is used in the current version.

Module	CoreDataStructure	KeyLogic
Quiz Game	Dictionary of Q&A	Case-insensitive answer check, score & percentage calculation, grade assignment (A–D).
File Organizer	List + extension rules	Classifies files into Images/Documents/Videos / Others by extension.
URL Shortener	Dictionary (short → long URL)	Random + string modules generate unique 6-character short codes; dictionary lookup for retrieval.
Password Manager	Dictionary (website → password)	Save, view, delete passwords; generates strong 10-character random passwords.

5 Performance Test:

Since these are lightweight, standalone command-line utilities rather than high-throughput industrial systems, the primary performance constraints identified were functional correctness, input validation, and logical accuracy rather than speed or memory. Each module was tested manually using representative and edge-case inputs to confirm it behaved as intended before being considered complete.

5.1 Test Plan/ Test Cases:

Module	Test Case	ExpectedResult	Status
Quiz Game	Enter correct answer (any case)	Marked correct, score incremented	Pass
Quiz Game	Enter in correct answer	Marked wrong, correct answer displayed	Pass
Quiz Game	Complete all 5 questions	Final score, percentage & grade shown	Pass
File Organizer	Files with known extensions (.jpg, .pdf, .mp4)	Correctly grouped under Images/Documents/Videos	Pass
File Organizer	File with unrecognized extension	Placed under Others	Pass
URL Shortener	Shorten a new long URL	Unique short.ly/code generated and stored	Pass
URL Shortener	Retrieve using a valid short URL	Correct original URL returned	Pass
URL Shortener	Retrieve using an unknown short URL	"URL not found" message shown	Pass
Password Manager	Save password for a website	Confirmation message, entry stored	Pass
Password Manager	View password for unsaved website	"No password found" message shown	Pass
Password Manager	Generate strong password	10-character random password with symbols shown	Pass

Password Manager	Delete an existing entry	Entry removed, confirmation shown	Pass
------------------	--------------------------	-----------------------------------	------

5.2 Test Procedure:

Each module was executed independently in the console (the File Organizer and Quiz Game were run and verified using an online Python compiler, as noted in the corresponding weekly reports). Test inputs covering both valid/expected cases and invalid/edge cases were entered manually for each menu option, and the printed output was checked against the expected behaviour listed in the test plan above.

5.3 Performance Outcome:

All 12 test cases across the four modules produced the expected output, with no unhandled exceptions during normal use. The main constraint identified was input validation. for example, the Quiz Game and Password Manager rely on the user entering exact menu options, and invalid input required explicit handling to avoid the program crashing; this was addressed by displaying an "Invalid choice" message and re-prompting rather than raising a runtime error.

6 My learnings:

Across the six weeks of this internship, I built a working proficiency in core Python programming while completing four independent, self-contained projects.

Week 1 (Quiz Game): Learned Python fundamentals, variables, data types, operators, loops, conditionals, and functions and applied them to build a scoring quiz engine, including handling answer validation and percentage/grade calculation.

Week 2 (File Organizer): Strengthened my understanding of loops, conditional statements, and lists, and learned how string methods (endswith()) can be used to classify data based on patterns such as file extensions.

Week 3 (URL Shortener): Gained practical experience with Python's random and string modules, and deepened my understanding of dictionaries for mapping and retrieving data reliably without duplication.

Week 4 (Password Manager): Learned to build a complete menu-driven application combining dictionaries, loops, and the random/string modules, while understanding the importance of secure password generation and structured data handling.

Beyond the technical syntax, this internship significantly improved my logical thinking, debugging discipline, and ability to test my own code critically before considering a task complete. Documenting each week's challenges and resolutions also improved my technical communication, an essential skill for any industry role. Overall, this experience has given me stronger confidence in independently designing, building, and testing Python applications, and it has directly reinforced the Python and problem-solving foundation I am building toward my goal of joining the IT industry.

7 Future work scope:

Add persistent storage (file-based JSON/CSV or a database) so that Quiz results, organized files, shortened URLs, and saved passwords are retained across sessions instead of only in memory.

Encrypt stored passwords (e.g., using the hashlib or cryptography module) instead of storing them in plain text, to make the Password Manager genuinely secure.

Extend the File Organizer to operate on real directories using the os and shutil modules, automatically moving files into folders rather than just categorizing a sample list.

Build a simple GUI (using Tkinter or a web front end with Flask) for all four tools to make them more accessible to non-technical users.

Add a larger, file-based question bank for the Quiz Game with support for multiple difficulty levels and question categories.

Deploy the URL Shortener as an actual web service with real redirect functionality, rather than a simulated in-memory mapping.